

How We Build This Product

A human guide to the AI Development Operating Protocol — who decides what, when work is allowed to start, and how it gets checked before it ships.

The one-sentence version. Nothing consequential gets built until a human says so, and nothing gets merged until a second reader has looked at it — the AIs plan, implement, and review, but the owner is the only one who authorizes and the only one who merges.

Protocol version 4.1 — adds state sync to v4

Adopted Task 219 (v4) · Task 220 (v4.1)

Authoritative source docs/Engineering/ai-development-operating-protocol.md

Generated 1 September 2026

The Cast

Four participants. Each is trusted with exactly one kind of thing, and deliberately not trusted with the others.

The protocol exists because AI can produce a lot of plausible work very quickly. Speed is only useful if the dangerous parts still pass through a human. So the roles are split so that **no single participant can both decide to do something and then do it.**

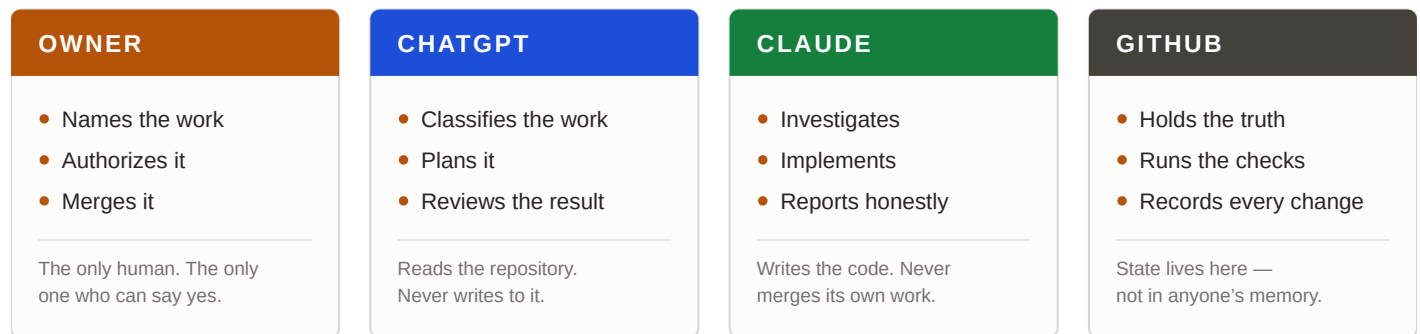


Figure 1. The separation of powers. Planning, implementing, and authorizing are three different jobs held by three different participants.

Why ChatGPT and Claude are kept apart

They are not two interchangeable assistants. They are deliberately given different information and different powers:



THE LOAD-BEARING RULE

Claude never merges its own work. Every change goes onto a branch and into a pull request, and the owner merges it. Branch protection is deliberately switched off in GitHub — so this is a convention followed, not a lock enforced. That is a considered trade-off: it keeps the owner from ever being locked out of their own project, at the cost of relying on discipline.

The First Question: What Kind of Work Is This?

Before anything else happens, the work gets sorted into one of four buckets. The bucket decides how much ceremony it needs.

This is the single most important step in the protocol. Everything downstream — whether a task number is assigned, whether you have to authorize it, whether a report gets written — follows from this one classification.

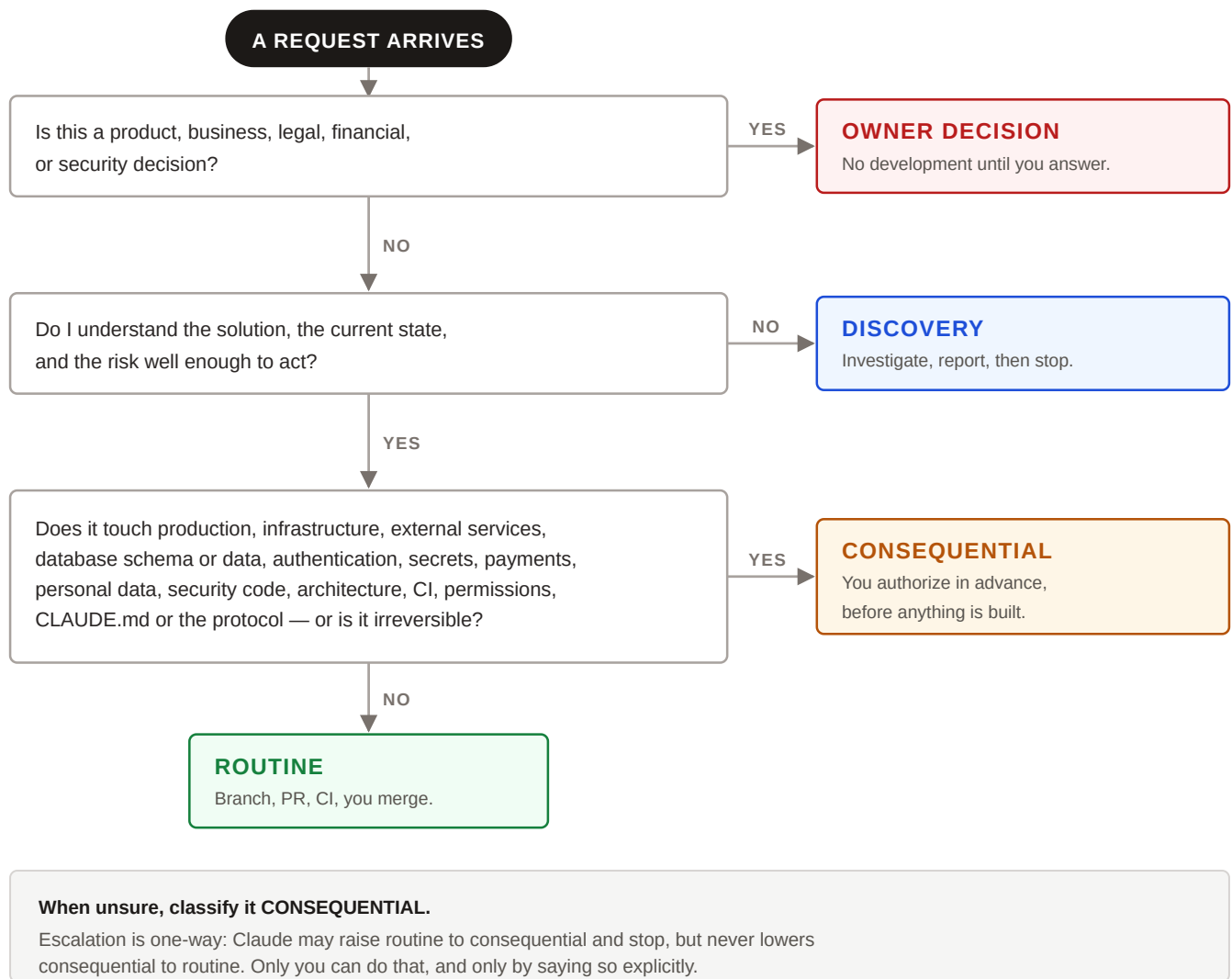


Figure 2. The classification decision. Asked in order, and answered before any work begins.

What the four buckets mean in practice

BUCKET	TYPICAL WORK	WHAT IT COSTS YOU
ROUTINE	Documentation, small bug fixes, tests, small UI changes, refactors inside one area, dependency bumps with no breaking changes.	One merge click. No task number, no task report — the pull request description <i>is</i> the report.
CONSEQUENTIAL	Anything touching production, money, identity, personal data, the database shape, the build pipeline, or the rules themselves. Anything irreversible.	You read a plan and explicitly authorize it before work starts. Then you read a report and merge.
DISCOVERY	The problem, the current state, or the risk isn't understood well enough yet.	You read findings and decide what, if anything, to do. No code is written.
OWNER DECISION	Product direction, business, legal, financial, security posture.	Only your judgment will do. Work stops until you answer.

THE TRAP THIS PREVENTS

The natural failure mode of a capable AI is to treat a big change as a small one because it *can* do it quickly. Ease is not the same as safety. A one-line change to a payment rule is trivial to type and expensive to get wrong — so difficulty never decides the bucket. **Blast radius and reversibility do.**

The Two Paths

Routine work takes the short road. Consequential work takes the long one. The only structural difference is where you sit in it.

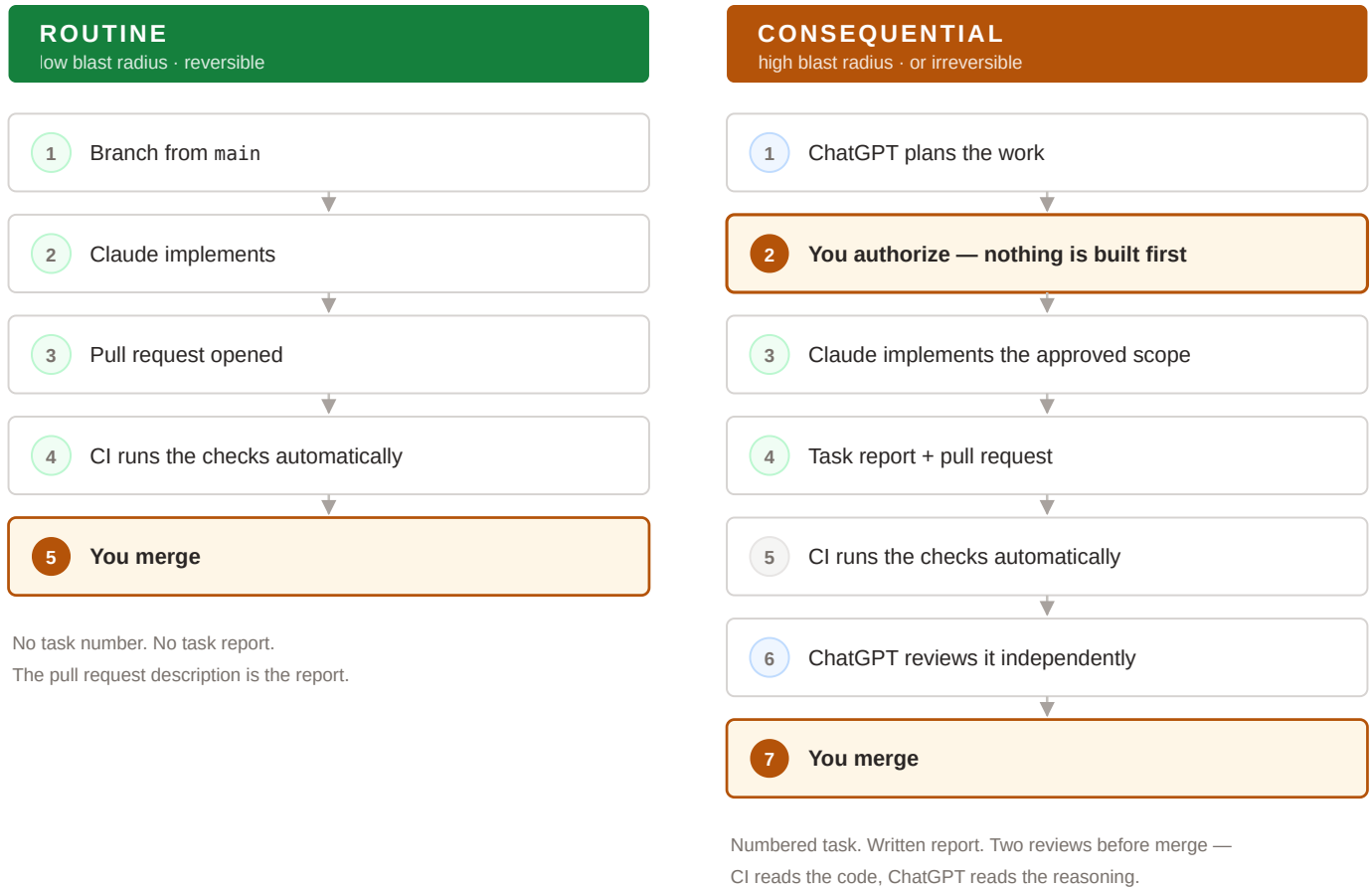


Figure 3. The two paths. Note where the amber blocks fall: every point at which the process waits for a human.

HOW TO READ THIS

Most work is routine, and routine work should feel light — if every typo fix demanded a plan and a report, the process would be abandoned within a week. The long path is reserved for the small number of changes that can actually hurt you.

Inside a Single Task

What actually happens between “go ahead” and “ready to merge.”

BEFORE	DURING	AFTER
■ Read the protocol and CLAUDE.md ■ Classify the request ■ Say the classification out loud ■ Assign the next task number ■ Present a plan — then wait	■ Implement only the agreed scope ■ Reuse existing patterns ■ Keep logic out of UI components ■ Out-of-scope finds go to the backlog, not into this change	■ Browser QA, sized to the change ■ Typecheck, lint, production build ■ Write the completion report ■ Update the task index + STATE ■ Commit, push, open the PR ■ STOP. Do not start the next task.

Figure 4. The task cycle from Claude’s side. The final step is a rule, not a suggestion.

Two habits that matter more than they look

Scope discipline. Claude will notice other problems while working — it always does. Those go into `docs/Product/backlog.md` or `docs/Product/ux-polish.md`, never into the current change. A pull request that quietly grew a second feature is a pull request nobody can review properly.

Stopping. After a task is reported, Claude stops and waits, even when the next step seems obvious and even when a previous report recommended it. A recommendation written last week is evidence about last week. A human decides what happens next.

THE HANDOFF BLOCK

Every finished task opens with a short block you copy straight into ChatGPT to start the review. It always names the real report path, the branch, and — as a factual count, not a courtesy — whether anything needs you to act outside the repository.

```
Task 220 is complete – Protocol v4.1: State Sync.

Report: docs/tasks/TASK-220/completion-report.md
Branch: task/220-state-sync
Owner action required: NO

Review the AI Development Workflow documentation and the report on GitHub.
```

Authorization

Two rules do most of the work here, and both are about the difference between *being able to* and *being allowed to*.

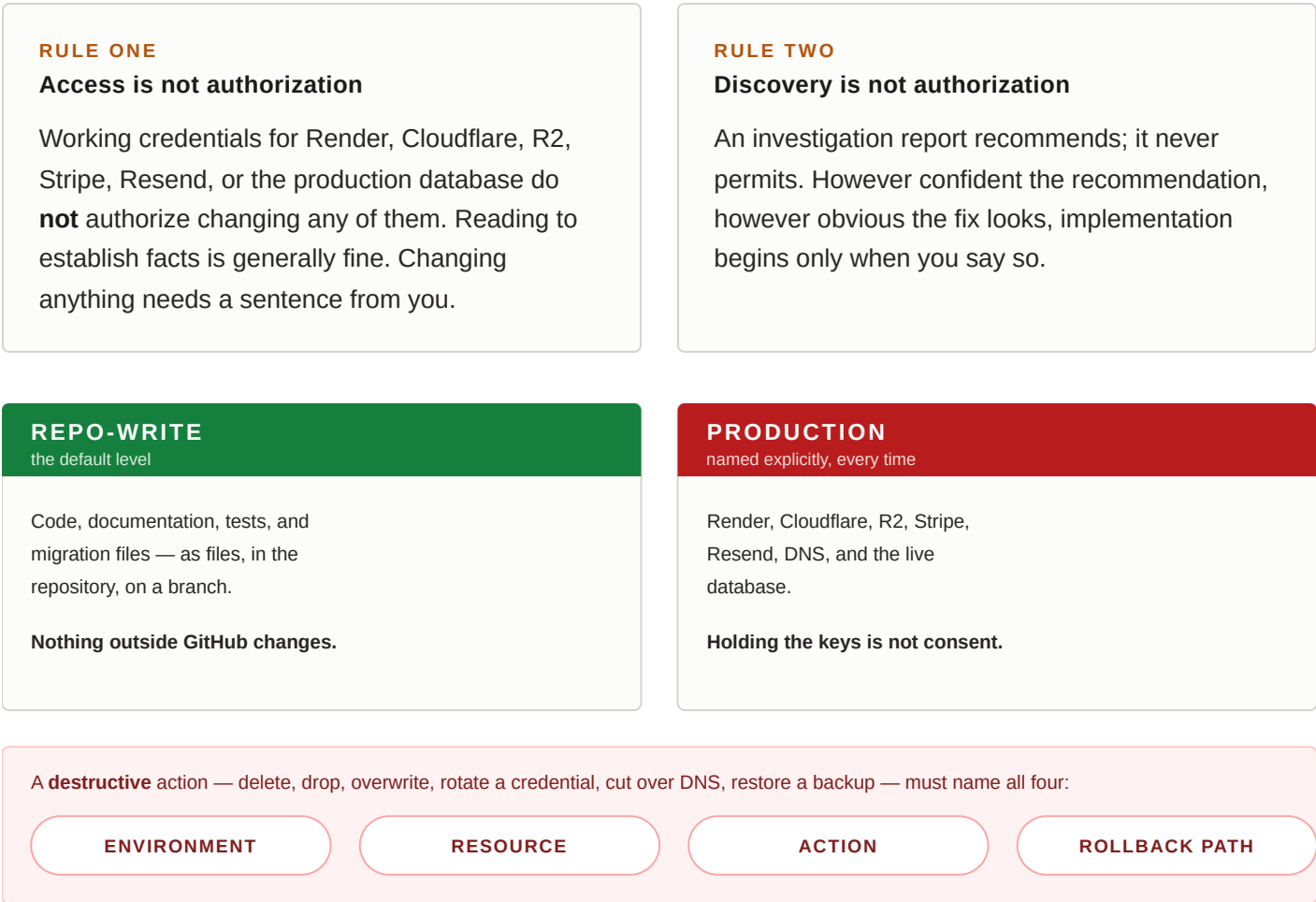


Figure 5. The two authorization levels. For destructive work, “there is no rollback” is an acceptable answer — leaving it unsaid is not.

How claims are labelled

A recurring failure with AI assistants is confident-sounding statements that were never checked. The protocol fixes this by requiring every factual claim to carry its provenance:

LABEL	MEANS	WEIGHT
Verified	Checked this session — a command was run, a query returned, an API answered.	Beats every document, including this one.
Historical	Comes from a past task report in <code>docs/tasks/</code> .	Evidence about the past. Never a statement about today.
Unverified	Assumed, inferred, or remembered.	Not a basis for action. The honest reply is “I don’t know; this needs checking.”

WHY THIS EARNS ITS KEEP

When documentation and reality disagree, Claude is required to trust what it verified, say so plainly, and flag the stale document — rather than quietly following either one. Several real defects in this project were found exactly that way: a document said one thing, a command said another, and the difference was the bug.

Keeping Everyone in Sync

The one addition in v4.1 — and the reason it was needed.

Version 4 said both AIs must work from repository files rather than conversational memory. It never said *how* ChatGPT gets re-synced after work lands. In practice it carried project facts across sessions in its head, and those facts went stale silently — because **a stale answer looks exactly like a fresh one**.

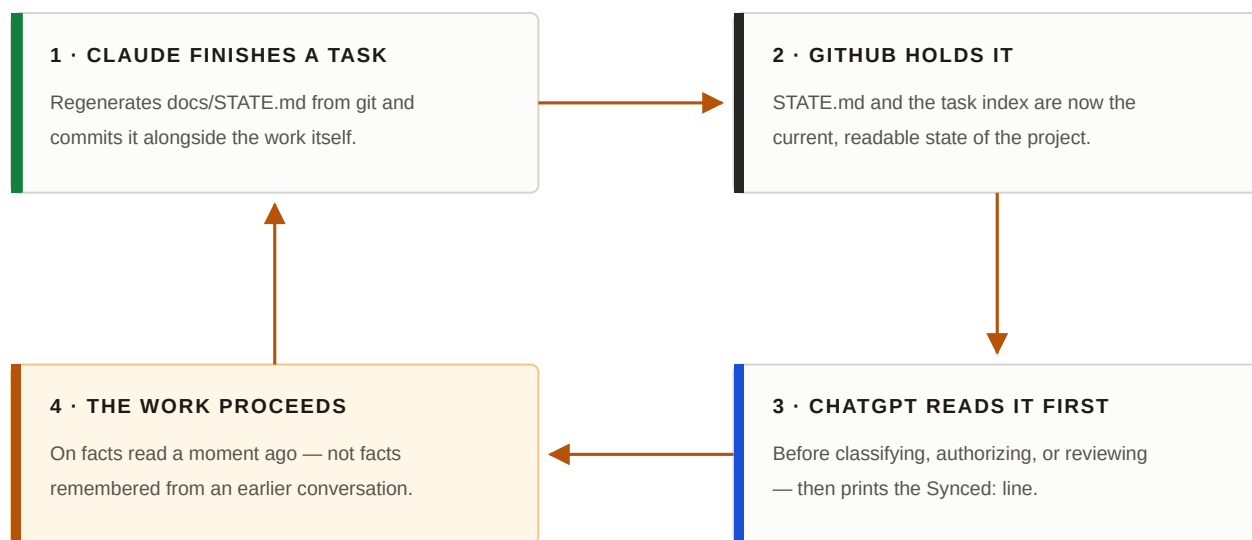


Figure 6. The sync loop. It runs once per task, and the only new obligation on each side is a single line.

What `docs/STATE.md` is — and is not

GENERATED STATE.md

Written by a script, overwritten every task. Holds only facts a command can produce: default branch, current commit, live branches, last task number, migration count, test count. Never hand-edited.

WRITTEN BY PEOPLE PROJECT-STATE.md

Direction, decisions, known gaps, priorities — everything requiring judgment. The rule of thumb: **if a fact needs a sentence of explanation, it does not belong in STATE.md.**

The generated file also names what it *cannot* know — deployed commit, CI result, external-service state are not derivable from the repository — so its silence is never mistaken for evidence.

The sync line

Every ChatGPT classification, authorization, and review now opens with one line naming what it read:

```
Synced: STATE @ Task 220 · main @ 48e8446 · CI green
```

BE CLEAR-EYED ABOUT THIS

Nothing can force ChatGPT to read the file. There is no technical control here and there cannot be one — its repository access is a fetch it chooses to make. The protocol says this outright rather than implying a guarantee it does not have.

What the line buys you is *visibility*. Skipping the read now produces either a missing marker or one naming a commit you can see is old, and a missing or stale marker is itself a review finding. The failure moves from silent to obvious. That is the achievable goal, not enforcement.

Your Part

Most of the protocol governs the AIs. Your side of it is four moments.

	MOMENT	WHAT YOU ACTUALLY DO
1	Name the work	Say what you want. You do not need to classify it — that comes back to you, and you can overrule it.
2	Authorize	Consequential work only. A sentence is enough, but it must name the environment, the resource, and the action. For anything destructive, the rollback path too.
3	Hand off	Paste the handoff block into ChatGPT to start the independent review.
4	Merge	You are the only one who does this. Neither AI merges its own work.

What this process does not protect you from

Stated plainly, because a process you trust more than it deserves is worse than no process:

GAP	WHY IT EXISTS
Branch protection is off	Deliberate — so you can never be locked out of your own repository. “Claude never merges” is therefore a convention followed, not a lock enforced.
CI is not a merge gate	It runs on every pull request and reports honestly, but nothing stops a merge over a red check except you noticing.
STATE.md can go stale	If a task forgets to regenerate it, the file quietly ages — and unlike the sync line, that staleness does not announce itself. CI regenerating it would close this; that is not built.
The sync read cannot be forced	Covered in Part Six. Visibility, not enforcement.
Nothing checks your recollection	The mechanism keeps <i>ChatGPT</i> current. It does nothing about drift in your own memory of where things stand.

Where the real documents live

FILE	AUTHORITATIVE FOR
ai-development-operating-protocol.md	Process, classification, authorization, review
CLAUDE.md	Repository rules, coding standards, Git, reporting
docs/STATE.md	What is true right now (generated)
docs/PROJECT-STATE.md	Direction, decisions, known gaps
docs/tasks/INDEX.md	Task number → artifacts
docs/tasks/TASK-<n>/	What happened, task by task

IN ONE PARAGRAPH

Sort the work by how badly it could go wrong. Let the cheap, reversible things move quickly. Make the expensive, irreversible things wait for a human sentence. Have the participant who wrote the code not be the participant who approves it. Write down what is true so nobody has to remember it. Everything else in this document is detail.

This guide is a human-readable summary. Where it differs from docs/Engineering/ai-development-operating-protocol.md, that file governs. Maker & Story · Protocol v4.1 · 1 September 2026.